Sesion 3: Azure DevOps

Test Plans · Pipelines · Load Testing · Automation — el mismo pipeline, otra nube

Certificacion 5: QA Cloud-Native - Automatizacion Inteligente con IA Generativa

Pre-requisito: subir la app | Azure DevOps | Test Plans | Pipelines | Load Testing | Automation | A tener en cuenta

Pre-requisito: subir / desplegar la app Tienda

Antes de automatizar y armar pipelines, necesitamos la app corriendo en algun lado

Que vamos a desplegar

La **app Tienda** del pre-requisito: un front (catalogo, detalle, carrito) + una API REST, todo en un solo proceso Node/Express. Los datos los genera Faker. Es la misma app que usaremos en TODAS las sesiones que siguen. Aqui vemos 4 formas de "subirla", de la mas simple a la mas parecida a produccion.

NIVEL 1 · LOCAL

En tu maquina

Node directo. Lo minimo para verla funcionar. Solo la ves tu, en localhost.

NIVEL 2 · DOCKER

En un contenedor

Empaquetada en una imagen que corre igual en cualquier maquina. Base de la Sesion 6.

NIVEL 3 · GITHUB

Codigo en la nube

No despliega la app, pero es el paso previo para que AWS/Azure la tomen en un pipeline.

NIVEL 4 · AWS

Accesible por internet App Runner

Una URL publica. Usamos App Runner: la forma mas simple de AWS, sin servidores que administrar.

Opcion 1 — Local (Node)

Lo mas rapido para verla. Requiere Node 20+.

PowerShell — desde prerequisito/app-tienda

```
npm install      # instala Express + Faker y genera los datos (postinstall)
npm start        # arranca en http://localhost:3000
```

Abre `http://localhost:3000/`. La API queda en `/api/productos` y la salud en `/api/health`. Para cambiar el puerto: `$env:PORT=8080; npm start`.

Opcion 2 — Docker (portable)

Empaqueta la app en una imagen. Corre igual en tu maquina, en un servidor o en la nube. Requiere Docker Desktop.

PowerShell — desde prerequisito/app-tienda

```
docker build -t app-tienda .          # construye la imagen
docker run -p 3000:3000 app-tienda    # corre el contenedor
```

El Dockerfile ya genera los datos con Faker durante el build y expone `/api/health` como healthcheck. Abre `http://localhost:3000/`.

Opcion 3 — Subir el codigo a GitHub

No despliega la app, pero es el requisito para que un pipeline (AWS o Azure) la tome. El `.gitignore` ya evita subir `node_modules` y los datos generados.

PowerShell — desde prerequisito/app-tienda

```
git init
git add .
git commit -m "App Tienda: base del laboratorio"
git branch -M main
git remote add origin https://github.com/<usuario>/app-tienda.git
git push -u origin main
```

Opcion 4 — AWS con App Runner (la mas simple para tener URL publica)

AWS App Runner toma tu repo de GitHub (o una imagen de contenedor), construye la app y la publica en una URL `https`, sin que administres servidores, redes ni balanceadores. Detecta el puerto y escala solo. Es la via mas corta para poner la app "en internet" en AWS.

🖍️ **EJERCICIO PRACTICO**

Ejercicio 0: publicar la app en AWS App Runner

1. Requisito: la app ya subida a GitHub (Opcion 3). Consola AWS → busca **App Runner** → **Create service**.
2. **Source:** elige **Source code repository** → conecta tu cuenta de GitHub (la primera vez autorizas la AWS Connector for GitHub) → selecciona el repo `<usuario>/app-tienda` y la rama `main`.
3. **Deployment settings: Automatic** (cada push a main vuelve a desplegar). Siguiente.
4. **Configure build:** Runtime = **Nodejs 18/20**. Build command: `npm install`. Start command: `node servidor/server.js`. Port: `3000`.
5. **Configure service:** nombre `app-tienda`, tamano minimo (0.25 vCPU / 0.5 GB basta). Deja el resto por defecto. Siguiente.
6. **Create & deploy.** App Runner construye y despliega (2-5 min). Al terminar te da una **Default domain**: una URL `https://xxxx.awsapprunner.com`.
7. Abre esa URL: veras el catalogo. Prueba `/api/health` y `/api/productos` sobre el mismo dominio. Esa URL es la que usaran las pruebas E2E cuando corran contra el ambiente desplegado.

💡 App Runner tambien acepta desplegar desde una **imagen de contenedor en ECR** (Container registry en vez de Source code). Si ya construiste la imagen Docker (Opcion 2), subirla a ECR y apuntarla desde App Runner es igual de simple y conecta con la Sesion 6.

⚠️ App Runner cobra por tiempo de ejecucion. Para clase: crea el servicio, haz la demo, y luego **Pause** o **Delete** el servicio para no acumular costo. Con Free Tier de AWS igual conviene apagarlo al terminar.

¿Cual elijo?

- **Solo verla / desarrollar:** Opcion 1 (local).
- **Demostrar portabilidad / preparar Sesion 6:** Opcion 2 (Docker).
- **Conectar un pipeline:** Opcion 3 (GitHub) es obligatoria como base.
- **Tener una URL publica en AWS rapido:** Opcion 4 (App Runner).

Para el resto de la sesion basta con tenerla corriendo (local o desplegada). Los ejercicios de Azure funcionan apuntando a cualquiera de las dos.

Azure DevOps: el mismo pipeline, otra consola

Lo que aprendiste en AWS se traslada casi directo a Azure

Puente desde el Dia 2 (AWS)

En AWS usamos CodeBuild (runner), CodePipeline (orquestador) e Inspector (seguridad). En Azure el concepto es identico; cambian los nombres:

- **Azure Pipelines** = CodeBuild + CodePipeline juntos (build + orquestacion en un YAML).
- **Azure Repos** = repositorio Git (o conectas GitHub directo).
- **Azure Test Plans** = gestion de casos de prueba (no tiene equivalente directo en AWS).
- **Azure Load Testing** = pruebas de carga gestionadas (JMeter en la nube).
- **Azure Automation** = runbooks y tareas programadas (como Lambda + EventBridge).

Que es Azure DevOps

Una suite de Microsoft para todo el ciclo de vida del software: repos, tableros (Boards), pipelines de CI/CD, gestion de pruebas (Test Plans) y artefactos. Se organiza en **Organizacion** → **Proyecto**. Todo lo de esta sesion vive dentro de un proyecto de Azure DevOps. Es gratis para equipos pequenos e incluye minutos de pipeline gratuitos.

🖍️ EJERCICIO PRACTICO

Ejercicio 1: crear la organizacion y el proyecto

1. Entra a `dev.azure.com` con tu cuenta Microsoft. Si es la primera vez, crea una **organizacion** (ej. `tu-nombre-qa`).
2. **New project**: nombre `tienda-qa`, visibilidad **Private**. Create.
3. Explora el menu lateral: **Boards, Repos, Pipelines, Test Plans**. Cada uno es un modulo. Los usaremos en orden.
4. Conecta el codigo: o subes la app a **Azure Repos**, o en Pipelines eliges **GitHub** como origen (recomendado si ya la subiste a GitHub en el pre-requisito).

Caso practico 1: Azure Test Plans

Gestion de casos de prueba: el "set de pruebas" formal antes de automatizar

Que es y para que sirve

Azure Test Plans organiza tus casos de prueba en una jerarquia: **Plan** → **Suite** → **Test Case**. Cada caso tiene pasos, datos esperados y resultado. Sirve para **trazabilidad** (qué se probó, quién, cuándo) y como puente entre negocio y automatizacion: los casos que aquí se definen son los que despues automatizaras con Playwright en la Sesion 4.

Nivel	Que es	Ejemplo en la tienda
Test Plan	Contenedor de todo lo que se prueba en una version	"Tienda v1 — Regresion"
Test Suite	Agrupa casos por funcionalidad	"Catalogo", "Carrito", "Pedidos"
Test Case	Un caso con pasos y resultado esperado	"Filtrar productos por categoria"

🖍️ **EJERCICIO PRACTICO**

Ejercicio 2: crear un plan y casos de prueba de la tienda

1. Menu **Test Plans** → **New Test Plan**. Nombre: `Tienda v1 - Regresion`. Create.
2. Dentro del plan, **New Suite** → **Static suite**. Crea tres: `Catalogo`, `Carrito`, `Pedidos`.
3. En la suite **Catalogo**, **New Test Case**. Titulo: `Filtrar productos por categoria`. En Steps escribe los pasos:
4. Paso 1: "Abrir la home" → Esperado: "Se ve el catalogo". Paso 2: "Elegir categoria Hogar en el filtro" → Esperado: "Solo se ven productos de Hogar". Save & Close.
5. Agrega 2 casos mas: en **Carrito** → `Agregar un producto al carrito`; en **Pedidos** → `Confirmar un pedido`. Estos son los casos que automatizaremos en la Sesion 4.
6. Ejecuta uno manualmente: selecciona el caso → **Run for web application** → marca cada paso como Pass/Fail mientras navegas la app. Al final, el caso queda con estado.

💡 Trazabilidad: cada Test Case puede vincularse a un Work Item (historia de usuario) y luego a un resultado de automatizacion. Asi negocio ve "la historia X tiene 3 casos, 3 automatizados, 3 en verde".

Modelado local (para reproducir sin cuenta)

Si no tienes cuenta Azure, el mismo set de casos lo dejamos escrito en el proyecto como referencia: mismos titulos y pasos. En la Sesion 4 cada uno de estos casos se convierte en un escenario BDD automatizado.

📘 DOCUMENTACION DE REFERENCIA

Los 3 tipos de Test Suite (saber cual usar)

Una suite agrupa casos dentro de un plan. Hay tres tipos y la eleccion afecta la trazabilidad:

- **Static suite:** agrupas casos a mano. Es la que usamos en el ejercicio (`Catalogo`, `Carrito`, `Pedidos`). Simple y directa.
- **Requirement-based suite:** queda ligada automaticamente a un work item del backlog (user story / PBI). Es la unica forma de lograr trazabilidad end-to-end "requisito → caso → resultado". Si negocio pide "demuestrame que la historia X esta probada", esta es la suite.
- **Query-based suite:** se llena sola con los casos que cumplen una consulta de work items (ej. todos los casos con tag `smoke`). Util para suites dinamicas que crecen sin mantenimiento manual.

Shared Steps y Shared Parameters (no repetir trabajo)

Dos work items que evitan duplicar casos cuando una suite crece:

- **Shared Steps:** una secuencia de pasos comun (login, navegar al catalogo, preparar datos) definida una sola vez y referenciada desde muchos casos. Si el flujo de login cambia, lo editas en un lugar.
- **Shared Parameters / parametrizacion:** insertas parametros en los pasos (`@categoria`, `@producto`) y provees una tabla de valores. El mismo caso corre N veces con distintos datos sin clonarlo. Es el equivalente manual al "data-driven test" que automatizaremos con Faker en la Sesion 4.

Resultados posibles al ejecutar un caso

Cuando corres un caso manual, cada caso termina con un **outcome**: `Passed`, `Failed`, `Blocked` (no se pudo probar por un impedimento) o `Not applicable`. Ojo con el reporte de progreso: los casos `Blocked`, `Not applicable` y en pausa NO cuentan para el `Passed%` ni el `Failed%`, por eso a veces el `Run%` no cuadra con la suma. Opcionalmente puedes activar el campo `Actual Result` para registrar que paso realmente cuando un paso falla.

Permisos: necesitas "Basic + Test Plans"

Crear y gestionar planes, suites, casos, configuraciones y parametros requiere el nivel de acceso **Basic + Test Plans**. Con solo **Basic** puedes ejecutar y ver graficos (pestañas Execute y Chart) pero NO veras la pestaña **Define** para autorear casos. Si el ejercicio no deja crear casos, es casi seguro un tema de nivel de acceso que el admin de la organizacion debe asignar.

Configurations: el mismo caso en varios entornos

Una **Test Configuration** describe el entorno de ejecucion (ej. "Chrome / Windows 11", "Firefox / Android"). Asignas configuraciones a un caso o suite para probar el mismo caso en varias combinaciones y que cada una tenga su propio resultado. Es la base de la matriz de compatibilidad.

Fuentes oficiales (Microsoft Learn): Que es Azure Test Plans · Tipos de test suite · Shared Steps · Parametrizar casos · Outcomes de ejecucion.
Contenido reformulado para cumplir con las restricciones de licencia.

Caso practico 2: Azure Pipelines

El CI/CD de Azure: un YAML versionado, igual que el buildspec de AWS

El azure-pipelines.yml

Es el corazon de la sesion. Un archivo YAML en la raiz del repo describe las **etapas (stages)** del pipeline. Nuestro pipeline tiene tres: **Build** → **SmokeTests** → **LoadTest**. Cada uno corre en un agente Linux hospedado por Azure. Es exactamente la misma idea del buildspec.yml del Dia 2.

Stage 1
BUILD
Node + npm install
genera datos Faker
→
Stage 2
SMOKE
arranca la app
smoke test API
→
Stage 3
LOAD
prueba de carga
+ quality gate SLA

azure-pipelines.yml (extracto)

```
trigger:
  branches: { include: [ main ] }

pool:
  vmImage: 'ubuntu-latest'

stages:
  - stage: Build
    jobs:
      - job: build
        steps:
          - task: NodeTool@0
            inputs: { versionSpec: '20.x' }
          - script: npm install
            workingDirectory: 'prerequisito/app-tienda'
  # ... SmokeTests y LoadTest a continuacion
```

🖍️ **EJERCICIO PRACTICO**

Ejercicio 3: crear el pipeline en Azure

1. Requisito: el repo con `azure-pipelines.yml` subido (Azure Repos o GitHub).
2. Menu **Pipelines** → **New pipeline**. Elige donde esta tu codigo: **GitHub** (autoriza la primera vez) o **Azure Repos Git**.
3. Selecciona tu repositorio. Azure detecta el YAML: elige **Existing Azure Pipelines YAML file** → ruta `/azure-pipelines.yml` .
4. **Run**. Veras los 3 stages ejecutarse en orden. Cada uno pasa a verde al terminar. Haz clic en un stage para ver los logs (igual que en CodeBuild).
5. **Reproducir en local (sin Azure):** arranca la app y corre los mismos pasos del pipeline en tu terminal:

PowerShell — reproducir el pipeline en local

```
# Terminal 1: arrancar la app (pre-requisito)
cd prerequisito/app-tienda
npm install ; npm start

# Terminal 2: correr los stages Smoke y Load
cd dia3/proyecto/tienda-azure
npm run smoke      # Stage SmokeTests
npm run carga      # Stage LoadTest (quality gate SLA)
```

💡 El pipeline se dispara solo con cada `git push` a `main` (por el bloque `trigger`). Igual que el webhook de CodePipeline en AWS.

📘 DOCUMENTACION DE REFERENCIA

Anatomia de un pipeline YAML (la jerarquia)

Un pipeline se lee de arriba hacia abajo en cuatro niveles anidados. Entender la jerarquia evita el 90% de los errores de indentacion:

- **stage** → division mayor del proceso ("Build", "Run tests", "Deploy"). Por defecto los stages corren **secuencialmente**: cada uno empieza solo cuando el anterior termina, salvo que uses `dependsOn` para cambiar el orden o paralelizar.
- **job** → dentro de un stage; es la unidad minima que se agenda a un agente. Un job corre en **un** agente y sus steps van en orden. Varios jobs pueden correr en paralelo.
- **step** → dentro de un job; cada paso es un `script` (un comando de shell) o un `task` (una accion reutilizable, ej. `NodeTool@0`).
- **pool** → define en que agente corre (ej. `vmImage: 'ubuntu-latest'` , un agente hospedado por Microsoft). Se puede fijar a nivel de pipeline, stage o job.

script vs task

Un **step** de tipo `script` ejecuta comandos como en tu terminal (es lo que hacemos con `npm install`). Un **task** es un componente empaquetado y versionado (`NombreTask@version`) que hace algo comun sin que escribas la logica: instalar una version de Node (`NodeTool@0`), publicar resultados de test, descargar artefactos, etc. Buscas tasks en el catalogo; no reinventas scripts.

Triggers: cuando corre el pipeline

La forma recomendada es declarar el disparador **dentro del YAML** (no por UI), para que viaje versionado con el repo:

- **CI trigger** (`trigger:`) → corre al hacer push a las ramas indicadas (en el ejercicio, `main`). Puedes filtrar por ramas, rutas o tags.
- **PR trigger** (`pr:`) → corre al abrir/actualizar un pull request. Valida el codigo *antes* de mezclarlo.
- **Scheduled / pipeline triggers** → por horario (cron) o al terminar otro pipeline. Si pones `trigger: none` el pipeline solo corre manualmente.

Agentes hospedados vs self-hosted

El `pool` con `vmImage` usa un **agente hospedado por Microsoft**: una VM limpia por corrida, sin que mantengas infraestructura, con minutos gratuitos para proyectos pequeños. Si necesitas software especial, hardware propio o acceso a una red privada, usas un **agente self-hosted** (una maquina tuya con el agente instalado). Mismo YAML, distinto donde corre.

Puente con AWS (Dia 2)

Donde el `buildspec.yml` tenia fases `install / pre_build / build / post_build` , aqui tienes `stages → jobs → steps` . La logica es la misma (preparar → probar → evaluar); cambian los nombres y que Azure junta "runner" y "orquestador" en un solo archivo.

Fuentes oficiales (Microsoft Learn): Definicion de pipeline · Stages · Jobs · Triggers · Referencia YAML completa.
Contenido reformulado para cumplir con las restricciones de licencia.

Caso practico 3: Azure Load Testing

Pruebas de carga: ¿aguanta la app cuando llegan muchos usuarios?

Que es

Un servicio gestionado que lanza carga masiva contra tu app usando **Apache JMeter** en la nube, sin que administres las maquinas generadoras. Mide throughput, latencias (p50, p90, p95, p99) y errores, y permite definir **criterios de fallo** (si p95 > X ms, la prueba falla). Ese criterio se convierte en un **quality gate** del pipeline.

Metrica	Que mide	SLA de ejemplo
Throughput	Peticiones por segundo que soporta	> 200 req/s
Latencia p95	95% de las peticiones responden por debajo de este tiempo	< 800 ms
Tasa de error	% de peticiones que fallan	< 1 %

La idea del Ejercicio 4 en una frase

Vas a **simular muchos usuarios usando la tienda a la vez**, medir que tan rapido responde y decidir **automaticamente** si ese rendimiento es aceptable segun un objetivo que tu fijas (el **SLA**). Ese "pasa / falla" automatico es un **quality gate**: si el rendimiento no cumple, el despliegue se detiene. La prueba local (`npm run carga`) es la version didactica; Azure Load Testing es el servicio real que hace lo mismo a gran escala.

Vocabulario minimo (para leer el reporte)

- **Usuario virtual / concurrencia:** un cliente simulado que pide paginas sin parar. Concurrencia 10 = 10 pidiendo a la vez.
- **Throughput:** peticiones por segundo que aguanto la app (mas alto = mejor).
- **p50 (mediana):** la mitad respondio mas rapido que ese tiempo = el usuario "tipico".
- **p95:** el 95% respondio por debajo de ese tiempo = la experiencia de la mayoria bajo carga. Es la metrica estrella (el promedio engaña).
- **Tasa de error:** % de peticiones que fallaron.
- **SLA:** el objetivo que te pones. Por defecto: `p95 < 800 ms` *y* `error < 1%`. Si ambos se cumplen → OK; si uno falla → FALLA.

🖍️ EJERCICIO PRACTICO

Ejercicio 4: prueba de carga y evaluacion del SLA (en local)

Necesitas **dos terminales**: una con la app corriendo y otra para lanzar la carga.

1 **Terminal 1 — arranca la app** y dejala corriendo:

PowerShell — Terminal 1 (desde prerequisito/app-tienda)

```
npm install
npm start        # queda en http://localhost:3000
```

2 **Terminal 2 — lanza la prueba de carga:**

PowerShell — Terminal 2 (desde dia3/proyecto/tienda-azure)

```
npm run carga
```

3. **Lee el reporte:** peticiones totales, throughput, p50, p95 y tasa de error. Al final evalua el SLA (`p95 < 800 ms` y `error < 1%`) y dice **CARGA OK** o **CARGA FALLIDA**. El script sale con codigo `0` si pasa y `1` si falla (ese exit 1 es lo que detendria un pipeline).

4. **Variante A — subir la intensidad.** Mas usuarios y mas tiempo; los numeros empeoran un poco porque exiges mas:

PowerShell — Terminal 2

```
$env:CONCURRENCIA="20"; $env:DURACION_S="10"; npm run carga
```

5. **Variante B — forzar el fallo (el momento clave).** Pones un SLA imposible para ver el gate en rojo:

PowerShell — Terminal 2

```
$env:SLA_P95_MS="5"; npm run carga

# Limpia la variable despues, o las siguientes corridas heredan el SLA de 5 ms:
Remove-Item Env:\SLA_P95_MS
```

6. Ahora el reporte dira **CARGA FALLIDA** y el script sale con error. Eso es exactamente lo que, dentro del pipeline, detendria el despliegue. **Esa es la leccion del ejercicio**, no los numeros concretos.

💡 Lo que de verdad hay que entender: una prueba produce un numero → se compara con un umbral → devuelve OK o FALLA → el pipeline respeta ese veredicto. Mismo patron que los tests funcionales (Dia 2) y el gate de seguridad. Aqui la metrica es el rendimiento.

📘 DOCUMENTACION DE REFERENCIA

Dos formas de definir la carga

- **URL-based test (quick test):** indicas una URL, cuantos usuarios virtuales y la duracion; Azure genera el plan por ti. Es lo mas rapido para empezar y lo que usamos en la demo apuntando a `/api/productos` .
- **JMeter o Locust:** subes tu propio script de **Apache JMeter** (`.jmx`) o **Locust** para escenarios realistas: varias peticiones encadenadas, endpoints no-HTTP, datos de entrada parametrizados, correlacion de tokens, etc. Es lo que usarias para un flujo completo (login → buscar → agregar al carrito → pagar).

Como se calcula la carga (y el costo)

El servicio reparte la carga en **test engine instances** (motores). Cada motor simula un bloque de usuarios virtuales; mas motores = mas usuarios en paralelo = mayor escala. Para un quick test basado en RPS, Azure deriva los usuarios virtuales con la formula:

Formula (docs oficiales)

```
# usuarios virtuales = (RPS objetivo * tiempo de respuesta ms) / 1000
```

Es decir, si quieres 200 req/s y cada peticion tarda 500 ms, necesitas ~100 usuarios virtuales concurrentes. Entender esto evita pedir "millones de usuarios" cuando en realidad lo que importa es el RPS objetivo y la latencia esperada.

Fail criteria: el quality gate, con precision

Un **fail criterion** es una condicion sobre una metrica de cliente. Defines uno o varios; si **al menos uno** se cumple, la prueba completa queda en estado `Failed` (ideal para detener un deploy). Cada criterio se arma con 4 piezas:

- **metrica:** `response_time_ms` , `latency` , `error` (%) o `requests_per_sec` .
- **agregado:** para tiempo de respuesta/latencia → `avg` , `min` , `max` , `p50` , `p90` , `p95` , `p99` ; para error → `percentage` ; para RPS → `avg` .
- **operador:** `>` o `<` .
- **valor umbral:** error en % (0-100), tiempo/latencia en ms.

Nuestro SLA local (`p95 < 800 ms` y `error < 1%`) es exactamente este modelo: dos criterios, agregado `p95` y `percentage` . Opcionalmente cada criterio tiene una `action` (`continue` por defecto, o `stop` para abortar la corrida).

Que significan los percentiles

Percentil	Lectura	Para que mirarlo
p50 (mediana)	El rendimiento "tipico"	La experiencia del usuario promedio
p95	95% responde por debajo de ese tiempo	La experiencia de la mayoria bajo carga (metrica estrella del SLA)
p99	Solo el 1% peor supera ese tiempo	El peor caso: picos y colas de latencia

El promedio (`avg`) engaña: unas pocas respuestas muy rapidas esconden picos lentos. Por eso el SLA se fija sobre p95/p99, no sobre el promedio.

Fuentes oficiales (Microsoft Learn): Que es Azure Load Testing · Definir fail criteria · Crear y gestionar tests (formula de VUs) · Test basado en JMeter.
Contenido reformulado para cumplir con las restricciones de licencia.

¿Y como se usa Azure Load Testing DE VERDAD, desde el portal?

Punto que confunde: Azure Load Testing **no vive dentro de Azure DevOps**. Es un recurso aparte que creas en `portal.azure.com`. Y como la carga la generan maquinas en la nube, necesita **una URL publica** (tu app desplegada: App Runner, App Service...). **No sirve `localhost`**, porque Azure no ve tu maquina.

Nivel 1 — Usarlo solo (demo desde el portal, sin pipeline)

1. En `portal.azure.com` → busca **Azure Load Testing** → **Create** → crea el recurso (nombre, grupo de recursos, region).
2. Dentro del recurso → **Tests** → **Create** → **Add a URL-based test** (lo simple: no necesitas script JMeter).
3. Pega la URL publica de tu app, por ej. `https://xxxx.awsapprunner.com/api/productos`.
4. Configura la carga: numero de **usuarios virtuales** y **duracion**.
5. En la pestaña **Test criteria** define el **fail criterion** (tu SLA): metrica `response_time_ms`, agregado `p95`, operador `>`, valor `800`. Significa "si p95 supera 800 ms, FALLA" = tu `SLA_P95_MS` local.
6. **Run.** Azure levanta los motores, genera la carga y muestra un dashboard con curvas de latencia, throughput y errores, y arriba un veredicto **Passed / Failed** segun tu criterio.

En una frase: es tu `carga-simple.js`, pero quien genera la carga es la infraestructura de Azure, y el SLA lo defines en una pantalla en vez de en una variable de entorno.

Nivel 2 — Unirlo al pipeline (aqui "se unen" las piezas del curso)

En tu `azure-pipelines.yml` agregas un **step** con la tarea oficial `AzureLoadTest@1`. Ese step le dice al servicio en la nube "corre este test". Si el servicio devuelve **Failed** por el criterio p95, el step falla; y un step fallido detiene el pipeline → **el despliegue se bloquea**. Es el mismo quality gate que tu `exit 1` local, pero ejecutado por Azure.

azure-pipelines.yml — step de carga en Azure (ejemplo)

```
- task: AzureLoadTest@1
  inputs:
    azureSubscription: 'mi-conexion-azure'   # service connection
    loadTestConfigFile: 'carga.yaml'         # define URL, usuarios, duracion y fail criteria
    resourceGroup: 'rg-qa'
    loadTestResource: 'mi-load-testing'
# Si el fail criterion (p95 > 800 ms) se cumple, el test queda Failed,
# el step falla y el pipeline se detiene: quality gate de rendimiento.
```

El puente mental: local vs Azure (es lo mismo, otra escala)

Pieza	Tu prueba local	Azure Load Testing
Quien genera la carga	tu PC (Node)	motores en la nube de Azure
A que le pega	`localhost:3000`	una URL publica (App Runner, App Service)
Donde defines el SLA	variable `SLA_P95_MS`	pantalla "Test criteria" (fail criterion)
El veredicto	`exit 0 / exit 1`	estado Passed / Failed
Como detiene el deploy	el pipeline respeta el exit code	el step `AzureLoadTest@1` falla y para el pipeline

Fuentes oficiales (Microsoft Learn): Crear y correr un test · Fail criteria · Integrar en CI/CD (AzureLoadTest@1).

Caso practico 4: Azure Automation

Tareas programadas y runbooks: QA que corre solo, sin que nadie lo dispare

Que es

Azure Automation ejecuta **runbooks** (scripts de PowerShell o Python) de forma programada o ante un evento, sin servidores que administrar. En QA sirve para tareas recurrentes: correr un smoke test cada hora contra produccion, generar datos de prueba cada noche, limpiar ambientes, o disparar el pipeline en un horario. Es el equivalente de Lambda + EventBridge de AWS.

Ejemplo aplicado a la tienda: smoke horario

Un runbook que cada hora llama a `/api/health` y `/api/productos` de la app desplegada, y si algo falla, envia una alerta. Asi te enteras de una caida en minutos, no cuando un cliente reclama. El script es el mismo `smoke-api.js` del Ejercicio, apuntado a la URL de produccion.

✏️ EJERCICIO PRACTICO

Ejercicio 5: simular el runbook de smoke horario (local)

1. El runbook de Azure ejecutaria nuestro smoke test contra la URL desplegada. Reproducelo apuntando `BASE_URL` a tu app (local o App Runner):

PowerShell — el "runbook" es este mismo comando

```
# Contra la app local
$env:BASE_URL="http://localhost:3000"; npm run smoke

# Contra la app desplegada en AWS App Runner
$env:BASE_URL="https://xxxx.awsapprunner.com"; npm run smoke
```

2. En Azure: crea una **Automation Account** → **Runbooks** → **Create** (tipo PowerShell). En el runbook, llama a las URLs con `Invoke-WebRequest` y valida el status.

3. Agrega un **Schedule**: cada 1 hora. Publica el runbook. A partir de ahi, Azure lo ejecuta solo y guarda el historial de cada corrida.

Runbook PowerShell (ejemplo para Azure Automation)

```
$base = "https://xxxx.awsapprunner.com"
$r = Invoke-WebRequest "$base/api/health" -UseBasicParsing
if ($r.StatusCode -ne 200) {
    throw "Smoke fallo: la app no responde (status $($r.StatusCode))"
}
Write-Output "Smoke OK: la app respondio 200 a las $(Get-Date)"
```

📘 DOCUMENTACION DE REFERENCIA

Tipos de runbook (elige segun el lenguaje y el caso)

- **PowerShell:** script de texto en PowerShell. Versiones soportadas hoy: **7.6**, **7.4** y **5.1**. Microsoft recomienda las LTS (7.6 o 7.4); 7.1 y 7.2 ya no tienen soporte. Es el tipo natural para nuestro smoke con `Invoke-WebRequest` .
- **Python:** script de texto en Python (version soportada actual: **3.10**; 2.7 y 3.8 quedaron fuera de soporte). Util si tu equipo de QA ya escribe en Python, como haremos en la Sesion 4.

- **PowerShell Workflow:** permite checkpoints y ejecucion en paralelo para procesos largos y resilientes.
- **Graphical / Graphical Workflow:** se arman visualmente a partir de cmdlets, sin escribir codigo; buenos para quien no programa.

Como se dispara un runbook

- **Schedule:** por horario recurrente (cada hora, diario, semanal, mensual). Es lo que usamos para el smoke horario. Puedes pasarle parametros al runbook por cada schedule.
- **Webhook:** una URL HTTP unica que, al recibir un POST, arranca el runbook. Deja que un servicio externo (un pipeline, un sistema de tickets) lo dispare.
- **Manual / on-demand:** desde el portal, PowerShell o la API de Azure.
- **Desde una alerta:** una alerta de Azure Monitor (metrica, log o activity log) puede ejecutar un runbook como accion de remediacion automatica.

Flujo de trabajo tipico

1. Crear la **Automation Account** en el portal.
2. **Crear o importar** el runbook (desde cero, un archivo, o la Runbook Gallery). Puedes editarlo desde el portal o con la **extension de VS Code** (crear, editar, probar, ver jobs, debug local).
3. **Probarlo** en el panel de Test antes de publicar.
4. **Publicar** (hasta que no publicas, el schedule no lo usa) y **vincular un Schedule** o webhook.
5. Azure guarda el **historial de jobs**: cada corrida con su salida, estado y errores.

Puente con AWS (Dia 2)

Azure Automation + Schedule equivale a **AWS Lambda + EventBridge**: codigo que corre solo, por horario o evento, sin servidores que administrar. El patron "monitoreo sintetico" (probar produccion 24/7 y alertar si falla) es el mismo en ambas nubes.

Fuentes oficiales (Microsoft Learn): Tipos de runbook y versiones · Iniciar runbooks (schedules) · Webhooks · Disparo por alerta · Gestionar runbooks.
Contenido reformulado para cumplir con las restricciones de licencia.

A tener en cuenta: AWS vs Azure

El concepto es el mismo; elige la nube por contexto, no por moda

Necesidad	AWS (Dia 2)	Azure (Dia 3)
Definir el CI/CD como codigo	buildspec.yml + CodePipeline	azure-pipelines.yml
Correr el build	CodeBuild	Azure Pipelines (agente)
Gestion de casos de prueba	(sin equivalente nativo)	Azure Test Plans
Pruebas de carga	(3rd party / propio)	Azure Load Testing
Tareas programadas	Lambda + EventBridge	Azure Automation
Publicar la app rapido	App Runner	App Service / Container Apps

Lo que construiste hoy

- Un **set de casos** formal en Test Plans (el puente a la automatizacion de la Sesion 4).
- Un **azure-pipelines.yml** con 3 stages, reproducible en local.
- Un **smoke test** que valida el despliegue y un **quality gate de carga** con SLA.
- Un **runbook** de smoke horario (Azure Automation).

Puente al Dia 4

Los casos que definimos en Test Plans (filtrar catalogo, agregar al carrito, confirmar pedido) los **automatizaremos con Playwright + Screenplay + BDD en Python**, con datos de Faker, e integraremos **MCP (IA)** al framework. La gestion manual de hoy se vuelve automatizacion inteligente manana.

Certificacion 5 - QA Cloud-Native: Automatizacion Inteligente con IA Generativa
Sesion 3: Azure DevOps — Test Plans · Pipelines · Load Testing · Automation

Documentacion oficial por tema (Microsoft Learn):

Test Plans | Pipelines | Referencia YAML | Load Testing | Fail criteria | Automation | Tipos de runbook